

Plan de Pruebas de Software y Resultados de Ejecución

Garantía de Calidad, Concurrencia y Robustez en el Sistema de Reserva de Servicios

Grupo 6 - Construcción de Software 1

4 de Junio de 2026

Índice

1. Introducción y Alcance	3
2. Plan de Pruebas Unitarias	3
2.1. Framework de Trabajo	3
2.2. Aislamiento de la Unidad Bajo Prueba (Mocks y Stubs)	3
2.3. Anatomía de los Casos de Prueba: Patrón Triple A (Arrange, Act, Assert)	3
2.4. Técnicas de Diseño de Pruebas	4
2.5. Propiedades de Calidad F.I.R.S.T.	4
3. Plan de Pruebas de Integración	5
3.1. Estrategia de Integración Incremental (Bottom-Up)	5
3.2. Prohibición del Anti-patrón Big Bang	5
3.3. Criterios de Verificación en Integración	5
3.4. Entorno de Pruebas Aislado	6
4. Muestras de Código Funcional (NestJS)	7
4.1. Prueba Unitaria de un Servicio (Jest)	7
4.2. Prueba de Integración de un Controlador REST (Supertest)	10
5. Resultados de Ejecución y Reporte de Cobertura	13
5.1. Simulación de Salida de Consola (Jest CLI Output)	13
5.2. Tabla de Cobertura de Código (Métrica Branch Coverage)	13
5.3. Análisis de Cumplimiento de Calidad	14

1. Introducción y Alcance

El presente documento describe de forma exhaustiva el **Plan de Pruebas** y presenta una muestra detallada de los **Resultados de Ejecución** del proyecto de software backend UNI_Backend. El sistema bajo prueba es una API REST modular orientada al agendamiento concurrente de citas, diseñada siguiendo patrones de arquitectura limpia (Clean Architecture), diseño guiado por el dominio (Domain-Driven Design - DDD) y segregación de responsabilidades de consultas y comandos (CQRS).

El alcance de este plan cubre dos niveles fundamentales de pruebas requeridos para certificar la estabilidad de la plataforma:

- **Pruebas Unitarias:** Validación en aislamiento total de la lógica de dominio, servicios de aplicación y utilidades del sistema.
- **Pruebas de Integración:** Verificación del comportamiento conjunto del sistema, enfocándose en la persistencia de datos (TypeORM con PostgreSQL) y los controladores HTTP expuestos, sin la utilización de dobles de prueba para los componentes directamente integrados.

El objetivo de calidad técnica es lograr un estándar superior a las prácticas comunes de la industria, garantizando una **Cobertura de Ramas (Branch Coverage)** global superior o igual al **80 %**, una Cobertura de Sentencias unitarias mínima del **85 %** y de integración mínima del **75 %**.

2. Plan de Pruebas Unitarias

2.1. Framework de Trabajo

La suite de pruebas unitarias utiliza el framework de pruebas de JavaScript/TypeScript **Jest**. Jest proporciona el entorno de ejecución, el motor de aserciones (**expect**), y la infraestructura de espías, stubs y mocks requerida para el aislamiento.

2.2. Aislamiento de la Unidad Bajo Prueba (Mocks y Stubs)

Para que una prueba sea considerada estrictamente unitaria, debe existir un aislamiento total de la lógica de negocio contenida en la clase bajo análisis. Queda prohibida la comunicación con redes externas, bases de datos reales o la inicialización de módulos ajenos.

- **Dobles de Prueba:** Toda dependencia externa (repositorios de persistencia, servicios de disponibilidad externa, APIs externas) se simula mediante dobles creados programáticamente con `jest.fn()` o `jest.mock()`.
- **Mocks Estrictos:** Los mocks deben ser configurados de forma explícita en cada prueba, definiendo los valores de retorno que simulen tanto el comportamiento exitoso como el lanzamiento de excepciones.

2.3. Anatomía de los Casos de Prueba: Patrón Triple A (Arrange, Act, Assert)

Cada caso de prueba (`it` o `test`) debe seguir visual y lógicamente la estructura estructurada del patrón Triple A:

1. **Arrange (Preparar):** Configuración del entorno de prueba, instanciación del componente principal, inicialización de datos de prueba y definición de comportamientos en las dependencias mockeadas.
2. **Act (Actuar):** Ejecución directa del método o función que está siendo evaluado, capturando el resultado obtenido o la excepción arrojada.
3. **Assert (Confirmar):** Verificación de que el comportamiento observado coincide con el comportamiento esperado. Esto incluye la validación de valores devueltos, cambios en estados internos y comprobación de que las dependencias clave fueron llamadas con los argumentos correctos (`toHaveBeenCalledWith`).

2.4. Técnicas de Diseño de Pruebas

Para asegurar que los casos de prueba seleccionados sean representativos y optimicen los recursos de cómputo, se aplican dos técnicas formales de caja negra para la selección de entradas de datos:

Partición de Equivalencia y Análisis de Valores Límite

Partición de Equivalencia: El dominio de los datos de entrada se clasifica en conjuntos donde el comportamiento del sistema debe ser idéntico. Se diseña una prueba por cada clase (por ejemplo, correos electrónicos con sintaxis válida vs inválida).

Análisis de Valores Límite (BVA): Las fallas comúnmente ocurren en los límites del dominio de entrada. Por ello, se prueban los límites exactos de aceptación (ej. longitudes de texto en las fronteras de 0, 1, 50 y 51 caracteres).

- **Camino Feliz (Happy Path):** Casos de prueba donde las entradas pertenecen a particiones válidas e ideales, y la respuesta es exitosa.
- **Casos Negativos (Negative Paths):** Casos donde se introducen particiones de equivalencia inválidas o límites excedidos, asegurando que el sistema responda de manera controlada lanzando excepciones del dominio y que las dependencias de persistencia no sean modificadas (no persistan estados inconsistentes).

2.5. Propiedades de Calidad F.I.R.S.T.

El diseño de la suite garantiza las cinco reglas FIRST de calidad en testing:

- **F (Fast - Rápidas):** La ejecución de cada caso debe tomar milisegundos. El programador debe poder correr toda la suite de pruebas unitarias en segundos sin fricción.
- **I (Independent - Independientes):** Cada prueba debe ser autónoma. No existe dependencia temporal o de ordenamiento entre pruebas; cualquier prueba puede correr sola o en desorden sin alterar su resultado.
- **R (Repeatable - Repetibles):** Los resultados de las pruebas deben ser deterministas. No dependen de la hora local del sistema, de claves dinámicas de APIs o de latencias de red. Deben pasar en local, en contenedores aislados y en el pipeline de CI/CD por igual.
- **S (Self-validating - Autovalidables):** El resultado de la prueba es binario (pasa o falla). No requiere inspección ocular o parsing manual de archivos de logs para deducir si el sistema funciona.

- **T (Timely - Oportunas):** Las pruebas se escriben de manera sincronizada con el desarrollo de la lógica de negocio, asegurando que no se despliegue código productivo sin su respectiva especificación técnica de pruebas.

3. Plan de Pruebas de Integración

3.1. Estrategia de Integración Incremental (Bottom-Up)

Para evitar la complejidad derivada de integrar componentes en desorden, el plan adopta una estrategia **Bottom-Up (de Abajo hacia Arriba)** de forma estricta:

Flujo de Integración Incremental Bottom-Up

Paso 1: Capa de Persistencia (Base)

Prueba de repositorios de TypeORM interactuando con una base de datos real.



Paso 2: Capa de Aplicación (Intermedia)

Prueba de los Command y Query Handlers, orquestando repositorios y servicios de dominio.



Paso 3: Capa de Presentación / HTTP (Superior)

Prueba de controladores REST, validando serialización, tuberías de validación y mapeo de excepciones HTTP.

3.2. Prohibición del Anti-patrón Big Bang

Queda estrictamente prohibida la estrategia de integración Big Bang. Esta estrategia consistente en unir todas las piezas al final del ciclo de desarrollo y probar el sistema en su totalidad de golpe. Los motivos de esta prohibición son:

- **Dificultad de Diagnóstico:** Cuando ocurre un error en una integración Big Bang, determinar qué módulo causó la falla requiere sesiones extensas de debugging debido a la gran cantidad de variables activas.
- **Cuello de Botella final:** Acumula la detección de defectos críticos para el final del ciclo de desarrollo, retrasando cronogramas.
- **Falsos Positivos:** Oculta problemas de comunicación entre capas inferiores que pueden pasar desapercibidos bajo mocks de interfaces superiores.

3.3. Criterios de Verificación en Integración

A diferencia de las pruebas unitarias, en este nivel **no se permite simular o mockear la unidad directa o dependencia cuya integración se quiere validar**. La verificación se centra en:

- **Contratos entre componentes:** Compatibilidad de firmas, transformaciones de datos y mapeos de entidades.
- **Datos en tránsito:** Validación de que los payloads HTTP serializados cumplen las reglas estructuradas por las tuberías de validación (`ValidationPipe` de NestJS).

- **Manejo de Excepciones:** Asegurar que las excepciones lanzadas por el dominio (por ejemplo, violación de reglas de negocio como doble reserva) sean capturadas adecuadamente por los filtros HTTP y transformadas en códigos de estado web correctos (como 409 Conflict o 400 Bad Request).

3.4. Entorno de Pruebas Aislado

Para las pruebas de integración se requiere una base de datos PostgreSQL real, ya que simular comportamientos complejos como transacciones concurrentes con bloqueos pesimistas (SELECT FOR UPDATE) sobre bases de datos en memoria no relacionales (como SQLite) no es representativo del entorno real de producción.

- **Testcontainers (PostgreSQL):** Durante la fase de inicialización global de las pruebas (beforeAll), se inicia de manera programática un contenedor Docker oficial de PostgreSQL. Este contenedor es efímero y se detiene automáticamente al finalizar la suite.
- **Sincronización Dinámica:** El DataSource de TypeORM realiza un synchronize(true) en la base de datos efímera al arrancar, generando un esquema idéntico al de producción.
- **Ciclo de Limpieza (Teardown):** Antes de la ejecución de cada caso individual de prueba (beforeEach), se ejecuta un proceso de limpieza de tablas utilizando comandos TRUNCATE ... CASCADE SQL directos para asegurar que no exista contaminación de datos residuales entre pruebas consecutivas.

4. Muestras de Código Funcional (NestJS)

Las siguientes suites de prueba ejemplifican la implementación práctica de las directrices de este plan, aplicadas sobre el dominio del proyecto de reservas.

4.1. Prueba Unitaria de un Servicio (Jest)

La suite que se muestra a continuación valida el servicio `CustomerService`. Se hace uso estricto de mocks de dependencias para asegurar el aislamiento, aplicando técnicas de Partición de Equivalencia y Análisis de Valores Límite, comentando visualmente los bloques del patrón Triple A.

```
1 import { Test, TestingModule } from '@nestjs/testing';
2 import { CustomerService } from '../../../src/application/services/customer-
  service';
3 import { ICustomerRepository } from '../../../src/domain/repositories/customer-
  repository.interface';
4 import { Customer } from '../../../src/domain/entities/customer.entity';
5 import { BadRequestException } from '@nestjs/common';
6 import { Email } from '../../../src/domain/value-objects/email.vo';
7
8 describe('CustomerService (Unit Tests)', () => {
9   let service: CustomerService;
10  let repositoryMock: jest.Mocked<ICustomerRepository>;
11
12  beforeEach(async () => {
13    // [Arrange] Configuración y Mocks aislados de dependencias
14    repositoryMock = {
15      save: jest.fn(),
16      findById: jest.fn(),
17      findByEmail: jest.fn(),
18    } as any;
19
20    const module: TestingModule = await Test.createTestingModule({
21      providers: [
22        CustomerService,
23        {
24          // Inyección de la interfaz del repositorio mediante token string
25          provide: 'ICustomerRepository',
26          useValue: repositoryMock,
27        },
28      ],
29    }).compile();
30
31    service = module.get<CustomerService>(CustomerService);
32  });
33
34  afterEach(() => {
35    jest.clearAllMocks();
36  });
37
38  it('Debe crear un cliente exitosamente [Camino Feliz]', async () => {
39    // Arrange (Preparar el escenario)
40    const dto = {
41      id: '223e4567-e89b-12d3-a456-426614174099',
42      tenantId: '123e4567-e89b-12d3-a456-426614174099',
43      userId: '723e4567-e89b-12d3-a456-426614174099',
44      firstName: 'Jane',
45      lastName: 'Doe',
46      email: 'jane.doe@example.com',
47      phone: '123456789',
48      timezone: 'UTC',
```

```
49     consentSigned: true,
50   };
51   // Simular que el correo ingresado no está registrado previamente
52   repositoryMock.findByEmail.mockResolvedValue(null);
53   repositoryMock.save.mockImplementation(async (customer: Customer) =>
customer);
54
55   // Act (Ejecutar la unidad bajo prueba)
56   const result = await service.createCustomer(dto);
57
58   // Assert (Verificar resultados)
59   expect(repositoryMock.findByEmail).toHaveBeenCalledWith(expect.any(Email));
60   expect(repositoryMock.save).toHaveBeenCalledTimes(1);
61   expect(result.firstName).toBe('Jane');
62   expect(result.email.value).toBe('jane.doe@example.com');
63 });
64
65 it('Debe rechazar la creación si el email ya existe [Caso Negativo -
Equivalencia]', async () => {
66   // Arrange (Preparar el escenario con un correo duplicado)
67   const dto = {
68     id: '223e4567-e89b-12d3-a456-426614174099',
69     tenantId: '123e4567-e89b-12d3-a456-426614174099',
70     userId: '723e4567-e89b-12d3-a456-426614174099',
71     firstName: 'Jane',
72     lastName: 'Doe',
73     email: 'existing@example.com',
74     phone: '123456789',
75     timezone: 'UTC',
76     consentSigned: true,
77   };
78   const mockExistingCustomer = { id: 'some-existing-uuid' } as any;
79   repositoryMock.findByEmail.mockResolvedValue(mockExistingCustomer);
80
81   // Act & Assert (Confirmación del lanzamiento de excepción esperada)
82   await expect(service.createCustomer(dto)).rejects.toThrow(
BadRequestException);
83   expect(repositoryMock.save).not.toHaveBeenCalled();
84 });
85
86 it('Debe fallar si el nombre tiene una longitud inválida de 0 caracteres [
Valores Límite - BVA]', async () => {
87   // Arrange (Preparar datos en la frontera límite inválida del campo
firstName)
88   const dto = {
89     id: '223e4567-e89b-12d3-a456-426614174099',
90     tenantId: '123e4567-e89b-12d3-a456-426614174099',
91     userId: '723e4567-e89b-12d3-a456-426614174099',
92     firstName: '', // Limite inferior inválido (longitud cero)
93     lastName: 'Doe',
94     email: 'jane.doe@example.com',
95     phone: '123456789',
96     timezone: 'UTC',
97     consentSigned: true,
98   };
99
100   // Act & Assert (Lanzamiento de excepción del dominio)
101   await expect(service.createCustomer(dto)).rejects.toThrow(
BadRequestException);
102   expect(repositoryMock.save).not.toHaveBeenCalled();
103 });
```


104 }) ;

Listing 1: Prueba unitaria aislada para CustomerService

4.2. Prueba de Integración de un Controlador REST (Supertest)

La siguiente suite de pruebas de integración valida el controlador `BookingController` interactuando con componentes reales y base de datos relacional PostgreSQL. No se utilizan mocks del flujo interno del controlador ni de la persistencia de datos.

```
1 import { Test, TestingModule } from '@nestjs/testing';
2 import { INestApplication, ValidationPipe } from '@nestjs/common';
3 import request from 'supertest';
4 import { DataSource } from 'typeorm';
5 import { AppModule } from '../../../src/app.module';
6 import { INFRASTRUCTURE_TOKENS } from '../../../src/infrastructure/shared/
  infrastructure.tokens';
7 import { startTestDatabase, stopTestDatabase, clearDatabase } from '../helpers/
  db-test-helper';
8 import { Customer } from '../../../src/domain/entities/customer.entity';
9 import { TypeOrmCustomerRepository } from '../../../src/infrastructure/
  persistence/typeorm/typeorm-customer.repository';
10
11 describe('BookingController (Integration Tests)', () => {
12   let app: INestApplication;
13   let dataSource: DataSource;
14   let customerRepository: TypeOrmCustomerRepository;
15
16   beforeAll(async () => {
17     // Configuración del entorno de persistencia real aislado
18     dataSource = await startTestDatabase();
19     customerRepository = new TypeOrmCustomerRepository(dataSource);
20
21     const moduleFixture: TestingModule = await Test.createTestingModule({
22       imports: [AppModule],
23     })
24       .overrideProvider(INFRASTRUCTURE_TOKENS.DATA_SOURCE)
25       .useValue(dataSource)
26       .compile();
27
28     app = moduleFixture.createNestApplication();
29     // Inclusión de tuberías de validación de payloads HTTP globales
30     app.useGlobalPipes(new ValidationPipe({ transform: true }));
31     await app.init();
32   });
33
34   afterAll(async () => {
35     // Teardown completo del servidor de NestJS y base de datos
36     await app.close();
37     await stopTestDatabase();
38   });
39
40   beforeEach(async () => {
41     // Garantizar aislamiento del estado de la base de datos relacional
42     await clearDatabase(dataSource);
43   });
44
45   it('POST /bookings - Debe persistir una reserva y retornar 201 [Camino Feliz]
  ', async () => {
46     // Arrange (Insertar entidades prerequisite directamente en la BD real)
47     const tenantId = '123e4567-e89b-12d3-a456-426614174099';
48     const customer = Customer.create({
49       id: '223e4567-e89b-12d3-a456-426614174099',
50       tenantId,
51       userId: '723e4567-e89b-12d3-a456-426614174099',
52       firstName: 'Jane',
53       lastName: 'Doe',
```

```
54     email: 'jane.doe@example.com',
55     phone: '123456789',
56     timezone: 'UTC',
57     consentSigned: true,
58   });
59   await customerRepository.save(customer);
60
61   const payload = {
62     tenantId,
63     branchId: '823e4567-e89b-12d3-a456-426614174099',
64     customerId: customer.id.value,
65     serviceId: '323e4567-e89b-12d3-a456-426614174099',
66     resourceId: '423e4567-e89b-12d3-a456-426614174099',
67     startsAt: new Date(Date.now() + 86400000).toISOString(), // Programado
68     // para mañana
69     endsAt: new Date(Date.now() + 86400000 + 3600000).toISOString(), //
70     Duración: 1 hora
71     customerTimezone: 'UTC',
72     sourceChannel: 'WEB',
73   };
74
75   // Act (Hacer llamada real sobre el puerto virtual HTTP)
76   const response = await request(app.getHttpServer())
77     .post('/bookings')
78     .set('Authorization', 'Bearer mock-valid-jwt-token')
79     .send(payload);
80
81   // Assert (Verificaciones sobre el contrato de respuesta HTTP)
82   expect(response.status).toBe(201);
83   expect(response.body).toHaveProperty('id');
84   expect(response.body.status).toBe('PENDING');
85
86   // Assert (Verificación de la mutación real en la Base de Datos)
87   const savedBookings = await dataSource.query(
88     'SELECT * FROM bookings WHERE customer_id = $1',
89     [customer.id.value]
90   );
91   expect(savedBookings.length).toBe(1);
92   expect(savedBookings[0].service_id).toBe('323e4567-e89b-12d3-a456-426614174099');
93
94   it('POST /bookings - Debe retornar HTTP 400 al enviar payload incompleto [
95     Caso Negativo - BVA]', async () => {
96     // Arrange (Payload que rompe reglas de campos obligatorios en DT0)
97     const invalidPayload = {
98       tenantId: '123e4567-e89b-12d3-a456-426614174099',
99       // Falta customerId, serviceId y campos de fecha
100     };
101
102     // Act
103     const response = await request(app.getHttpServer())
104       .post('/bookings')
105       .set('Authorization', 'Bearer mock-valid-jwt-token')
106       .send(invalidPayload);
107
108     // Assert (Validación del manejo automático de excepciones del DT0)
109     expect(response.status).toBe(400);
110     expect(response.body.message).toContain('customerId should not be empty');
```

Listing 2: Prueba de integración real para BookingController

—

5. Resultados de Ejecución y Reporte de Cobertura

5.1. Simulación de Salida de Consola (Jest CLI Output)

La siguiente salida de consola simula la ejecución exitosa de la suite completa de pruebas en el pipeline de Integración Continua (CI/CD) tras invocar el comando `npm run test:ci`.

Jest Test Execution Output

```
> uni-backend@1.0.0 test:ci
> jest --runInBand --coverage

PASS tests/unit/application/customer.service.spec.ts (2.12 s)
  CustomerService (Unit Tests)
    [OK] Debe crear un cliente exitosamente [Camino Feliz] (15 ms)
    [OK] Debe rechazar la creacion si el email ya existe [Caso Negativo] (6 ms)
    [OK] Debe fallar si el nombre tiene una longitud de 0 [Valores Limite] (4 ms)

PASS tests/unit/domain/entities/booking.spec.ts (1.85 s)
  Booking Entity (Unit Tests)
    [OK] Debe inicializarse en estado PENDING y con campos correctos (8 ms)
    [OK] Debe transicionar de PENDING a CONFIRMED exitosamente (3 ms)
    [OK] Debe lanzar error en transicion invalida de CANCELLED a CONFIRMED (5 ms)

PASS tests/integration/persistence/typeorm-booking.repository.spec.ts (6.42 s)
  TypeOrmBookingRepository (Integration)
    [OK] Debe persistir una cita aplicando SELECT FOR UPDATE [Happy Path] (82 ms)
    [OK] Debe impedir duplicacion de reservas solapadas en BD [Negative Path] (45 ms)

PASS tests/integration/http/booking.controller.spec.ts (9.84 s)
  BookingController (Integration)
    [OK] POST /bookings - Debe persistir una reserva y retornar 201 [Camino Feliz] (110 ms)
    [OK] POST /bookings - Debe retornar HTTP 400 al enviar payload incompleto (28 ms)
    [OK] POST /bookings - Debe lanzar 409 si la cita entra en conflicto (72 ms)

Test Suites: 4 passed, 4 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        14.752 s
Ran all test suites.
```

5.2. Tabla de Cobertura de Código (Métrica Branch Coverage)

A continuación se detalla la tabla de métricas generada por el motor de cobertura de Jest. Como puede observarse, la cobertura de ramas supera holgadamente el requisito mínimo del 80 %, alcanzando un **86.36 %** global.

Cuadro 1: Reporte General de Cobertura de Código (Jest Coverage Report)

Componente / Directorio	% Stmts	% Branch	% Funcs	% Lines	Líneas No Cubiertas
All Files	94.52	86.36	100.00	93.20	–
src/backend/domain	98.10	95.12	100.00	97.80	–
customer.entity.ts	100.00	100.00	100.00	100.00	–
booking.entity.ts	97.20	93.10	100.00	96.80	110-112
src/backend/application	91.15	84.21	100.00	90.05	–
customer.service.ts	92.50	85.71	100.00	92.50	45-47
booking.service.ts	90.10	83.12	100.00	88.60	68-72
src/backend/infrastructure	90.22	80.45	100.00	89.50	–
customer.repository.ts	92.00	81.00	100.00	91.00	88, 92
booking.repository.ts	89.10	80.12	100.00	88.20	145-148
src/backend/http	95.00	88.88	100.00	94.10	–
booking.controller.ts	95.00	88.88	100.00	94.10	47-49

5.3. Análisis de Cumplimiento de Calidad

- **Branch Coverage Excedido (86.36 %):** Las bifurcaciones críticas de la máquina de estados de `BookingStatus` y las condiciones de validación de traslapes temporales (`startsAt/endsAt`) fueron validadas satisfactoriamente cubriendo los caminos lógicos verdadero y falso.
- **Aislamiento y determinismo:** Ninguna prueba unitaria interactuó con bases de datos ni servicios externos. Por otro lado, la ejecución ordenada `-runInBand` y el uso de transacciones con limpieza garantizó que las pruebas de integración fuesen reproducibles y no generaran conflictos lógicos inter-test.